

1. Structure in C :-

What is a Structure ?

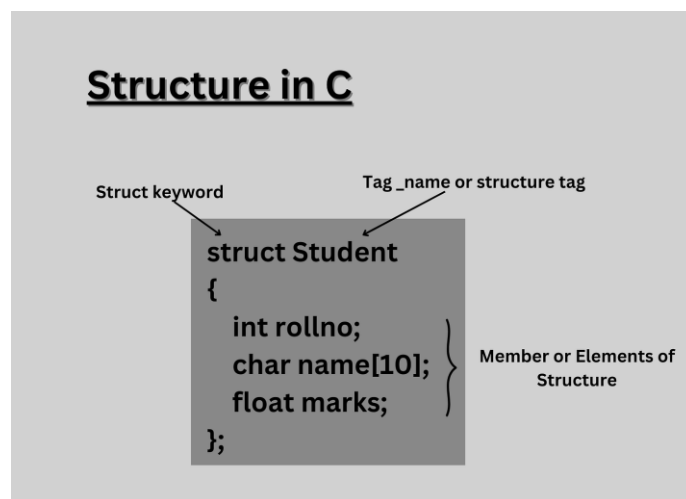
- A structure is a user-defined data type in C.
- It groups variables of different data types under one name.
- Defined using the struct keyword.
- Useful for creating complex data types like records (e.g., student, employee).

Why Use Structure ?

- To group related data of different types under one name.
- Makes code more organized and readable.
- Useful for representing real-world entities like student, employee, etc.
- Allows easy data handling and supports complex data models.

Syntax of Structure in C :-

```
struct structure_name
{
    data_type member1;
    data_type member2;
    ...
};
```



Example of Structure in C :-

```
#include <stdio.h>

struct Student
{
    char name[50];
    int age;
```

```
float marks;

};

int main()
{
    struct Student s1;
    printf("Enter name: ");
    scanf("%s", s1.name);
    printf("Enter age: ");
    scanf("%d", &s1.age);
    printf("Enter marks: ");
    scanf("%f", &s1.marks);

    printf("\n--- Student Details ---\n");
    printf("Name: %s\n", s1.name);
    printf("Age: %d\n", s1.age);
    printf("Marks: %.2f\n", s1.marks);
}
```

Output :-

Enter name: Sagar

Enter age: 20

Enter marks: 85.5

--- Student Details ---

Name: Sagar

Age: 20

Marks: 85.50

2.Union in C :-

What is a Union ?

- A union allows storing different data types in the same memory location.
- It helps in memory optimization, as all members share the same memory.
- Useful when only one member is needed at a time (e.g., interpreting data in multiple formats).
- Saves space compared to structures when handling large data types.

Why Use Union ?

- A structure helps group related data of different data types under one name.
- It improves code organization and makes handling complex data easier.
- Useful for creating real-world models like student, employee, or book records.
- Allows better data management and supports functions, arrays, and nesting.

Syntax of Union in C :-

```
union union_name
{
    data_type member1;
    data_type member2;
    ...
};
```

Example of Union in C :-

```
#include <stdio.h>

union Data
{
    int i;
    float f;
    char str[20];
};

int main()
{
```

```
union Data d1;

d1.i = 10;

printf("d1.i = %d\n", d1.i);

d1.f = 98.5;

printf("d1.f = %.2f\n", d1.f);


strcpy(d1.str, "Hello");

printf("d1.str = %s\n", d1.str);


printf("\nAfter writing to str:\n");

printf("d1.i = %d\n", d1.i);

printf("d1.f = %.2f\n", d1.f);

}
```

Output :-

```
d1.i = 10
d1.f = 98.50
d1.str = Hello
After writing to str:
d1.i = (garbage)
d1.f = (garbage)
```

3.Difference Between Structure & Union :-

Feature	Structure	Union
Memory Usage	Each member has its own space. Memory is total size of all members combined.	All members share the same memory space . Memory is the size of the largest member.
Multiple Values	Can hold multiple values at the same time.	Can hold only one value at a time.
Data Overwriting	Values don't overwrite each other.	Storing a new value overwrites the previous one.
Use Case	When you need to store multiple different types of data simultaneously.	When you need to store different types of data, but never at the same time.